

Professional Web Development by Miguel Angel Gutierrez Baca

Student name: Axel Antonio Martinez Aguirre

Secure Software Development Life Cycle (SSDLC)

Security-Focused Software Development Report

Secure Software Development Life Cycle (SSDLC)

The Secure Software Development Life Cycle (SSDLC) integrates security activities throughout the software development life cycle. Security is treated as a continuous responsibility beginning with requirements and continuing through design, coding, testing, deployment, and maintenance.

This report focuses specifically on the security practices, objectives, and activities applied during the six core SSDLC phases.

1. Requirements

Security Objectives

Establish what the software must protect and which security properties it must provide, including confidentiality, integrity, availability, authentication, authorization, privacy, and accountability.

Risk Assessment

Identify assets, threats, vulnerabilities, business impact, and attack scenarios. Document and prioritize risks according to likelihood and impact.

Specific Security Practices

- Define security requirements for authentication, authorization, privacy, encryption, logging, data protection, and availability.
- Identify and classify sensitive assets.
- Perform risk assessment and document threats, vulnerabilities, impacts, and mitigations.
- Define measurable security acceptance criteria.
- Identify applicable legal, regulatory, contractual, and organizational security obligations.

2. Design

Threat Modeling

Analyze how an attacker could misuse the planned system. Identify trust boundaries, entry points, data flows, sensitive assets, and potential threats. STRIDE is one model that can organize common threat categories.

Secure Architecture Design

Incorporate security controls before implementation, including strong authentication, least-privilege authorization, secure sessions, encryption, input-validation boundaries, secrets management, network segmentation, and secure error handling.

Security Design Review

Review requirements and threat-model findings against architecture decisions. High-risk decisions should have documented mitigations and verification criteria.

Specific Security Practices

- Map data flows, trust boundaries, external interfaces, privileged operations, and sensitive assets.
- Perform threat modeling and document mitigations.
- Apply least privilege, defense in depth, secure defaults, and fail-safe design.
- Design authentication, authorization, encryption, session management, and secrets handling.
- Conduct security architecture reviews before implementation.

3. Development (Coding)

Secure Coding Standards

Follow secure coding standards appropriate to the programming language and application. Practices include input validation, prevention of injection, parameterized queries, safe authentication and authorization, avoiding hard-coded secrets, protecting sensitive data, and safe error handling.

Static Analysis

Static Application Security Testing (SAST) analyzes source code without executing the application and can identify patterns associated with injection, insecure APIs, weak cryptography, hard-coded credentials, unsafe data flows, and access-control problems.

Dependency and Secret Checks

Check third-party libraries for known vulnerabilities and use automated secret detection to identify accidentally committed keys, passwords, tokens, or certificates.

Developer Security Practices

Track and remediate security findings. Peer code review should include security considerations, especially for authentication, authorization, cryptography, file handling, database access, and integrations.

4. Testing

Dynamic Analysis

Dynamic Application Security Testing (DAST) evaluates the running application and can identify authentication problems, insecure sessions, injection issues, exposed endpoints, and security misconfigurations.

Vulnerability Scanning

Automated scanners examine applications, dependencies, hosts, containers, and infrastructure for known weaknesses or insecure configurations. Findings should be validated, prioritized, assigned, and tracked.

Penetration Testing

Authorized penetration testing uses controlled attack techniques to determine whether vulnerabilities can be exploited and what impact they could have. Testing should have defined scope and rules of

engagement.

Security Verification

Security tests should connect to the requirements and threat model. High-risk findings should be remediated or formally accepted before release, and regression testing should verify fixes.

5. Deployment

Secure Configuration

Use secure configuration baselines. Disable or restrict unnecessary services, ports, accounts, permissions, and features. Default passwords and credentials must not remain in production.

Environment Hardening

Harden servers, containers, databases, cloud resources, networks, and supporting services. Apply least privilege, restrict administrative interfaces, and protect production secrets.

Secure Release Controls

Protect build artifacts, source code, credentials, and deployment permissions. Integrate security checks into CI/CD pipelines.

Logging and Monitoring Preparation

Configure useful security logs and monitoring for authentication failures, privilege changes, suspicious requests, and administrative actions before or during release.

6. Maintenance

Patch Management

Monitor vendor advisories, vulnerability databases, security notifications, and internal findings. Evaluate, prioritize, test, and deploy security patches within appropriate time frames.

Continuous Monitoring

Monitor applications and infrastructure for suspicious activity, abnormal behavior, authentication anomalies, configuration changes, and indicators of compromise.

Vulnerability Management

Periodically scan deployed systems and dependencies, track newly discovered vulnerabilities, and verify remediation. Reassess previously accepted risks when systems or threats change.

Incident Response and Lessons Learned

Contain incidents, investigate causes, remediate affected components, and document lessons learned. Use findings to improve requirements, threat models, coding standards, testing, and deployment controls.

Security Activities Across the SSDLC

Security in SSDLC is cumulative: requirements establish what must be protected; design determines how protections work; development implements them; testing verifies effectiveness; deployment preserves secure configuration; and maintenance keeps the system secure as threats and dependencies change.

Phase	Primary Security Focus	Typical Security Activities
Requirements	Define and prioritize security needs	Risk assessment, security requirements, asset identification, security acceptance criteria
Design	Prevent weaknesses in architecture	Threat modeling, secure architecture, trust boundaries, least privilege, security review
Development	Implement secure controls	Secure coding, SAST, dependency checks, secret detection, code review
Testing	Find and validate vulnerabilities	DAST, vulnerability scanning, penetration testing, regression testing
Deployment	Protect production	Secure configuration, hardening, access control, secrets management, secure CI/CD
Maintenance	Keep security effective	Patch management, monitoring, vulnerability management, incident response

Conclusion

The SSDLC applies security continuously rather than limiting it to a final security test. Effective security begins with risk-based requirements, continues through threat-informed architecture and secure coding, and is verified through dynamic analysis, vulnerability scanning, and authorized penetration testing. Secure deployment and hardening protect the production environment, while patch management and continuous monitoring address new vulnerabilities and changing threats.

Student Submission Checklist

■	All 6 phases (Requirements, Design, Development, Testing, Deployment, Maintenance) are included.
■	The specific security practices for each phase are clearly explained.
■	The report focuses on security, not just general development steps.
■	Each phase has its own clear heading.
■	The document has been proofread for clarity and errors.
	TOTAL: 100/100

Checked by: Victoria Lazo Ramirez

End of Report